

Anexo 19 — Calibración inicial de actuadores

Archivo: Scripts/calibracion.py

Para que sirvió:

Trabajo previo a la etapa de RL: búsqueda en grilla paralela (multiprocessing) que barre kp, kv, damping y gear de cada actuador evaluando estabilidad, sobreimpulso y precisión.

```
1 | import multiprocessing as mp
2 | import mujoco
3 | import numpy as np
4 | import matplotlib.pyplot as plt
5 | import pandas as pd
6 | from functools import partial
7 | import time
8 | XML_PATH = "Cuerpo\DUM4.xml"
9 |
10 | def search_one_actuator(act_name, kp_values, kd_values, damping_values, gears,
11 |                        target_dict, sim_time, error_threshold):
12 |     """Worker independiente: carga su propia copia del modelo."""
13 |     # Carga silenciosa del modelo
14 |     model = mujoco.MjModel.from_xml_path(XML_PATH)
15 |     data = mujoco.MjData(model)
16 |
17 |     act_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, act_name)
18 |     joint_id = model.actuator_trnid[act_id, 0]
19 |     dof_id = model.jnt_dofadr[joint_id]
20 |     qpos_adr = model.jnt_qposadr[joint_id]
21 |
22 |     # El valor objetivo específico para este actuador
23 |     target = target_dict[act_name]
24 |
25 |     best_mse = float('inf')
26 |     winner = None
27 |     EQUALITY_FINGERS = {
28 |         "act_LeftLever_Slider": ["LeftFingerTop_joint", "LeftFingerBot_joint"],
29 |         "act_RightLever_Slider": ["RightFingerTop_joint", "RightFingerBot_joint"],
30 |     }
31 |
32 |     # Resolvé los qpos_adr de los fingers vinculados (solo si aplica)
33 |     finger_ids = {}
34 |     if act_name in EQUALITY_FINGERS:
35 |         for jname in EQUALITY_FINGERS[act_name]:
36 |             jid = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, jname)
37 |             finger_ids[jname] = model.jnt_qposadr[jid]
38 |     # Flush=True asegura que el print salga del proceso hijo a la terminal principal
39 |     print(f"[START] {act_name}", flush=True)
40 |
41 |     for g in gears:
42 |         for kp in kp_values:
43 |             print(f'{act_name} prueba {kp}', flush=True)
44 |             for kd in kd_values:
45 |                 for d in damping_values:
46 |                     model.actuator_gear[act_id, 0] = g
47 |                     model.actuator_gainprm[act_id, 0] = kp
48 |                     model.actuator_biasprm[act_id, 1] = -kp
49 |                     model.actuator_biasprm[act_id, 2] = -kd
50 |                     model.dof_damping[dof_id] = d
51 |
52 |                     mujoco.mj_resetData(model, data)
53 |                     temp_qpos = []
54 |                     while data.time < sim_time:
55 |                         # if data.time < 1.5:
56 |                         data.ctrl[act_id] = target
57 |                         mujoco.mj_step(model, data)
58 |                         temp_qpos.append(data.qpos[qpos_adr])
59 |                         final_pos = temp_qpos[-1]
60 |                         mse = np.mean((np.array(temp_qpos) - target)**2)
61 |
62 |                         if abs(final_pos - target) <= error_threshold:
63 |                             if mse < best_mse:
64 |                                 best_mse = mse
65 |                                 winner = {'kp': kp, 'kd': kd, 'd': d, 'g': g, 'final_val': final_pos}
66 |
67 |                     # else:
68 |                     #     data.ctrl[act_id] = 0.0
69 |                     #     mujoco.mj_step(model, data)
70 |                     #     temp_qpos.append(data.qpos[qpos_adr])
71 |
72 |                     #     final_pos = temp_qpos[-1]
73 |                     #     mse = np.mean((np.array(temp_qpos) - 0.0)**2)
74 |
75 |                     #     if abs(final_pos - 0.0) <= error_threshold:
```

```

75 |             #         if mse < best_mse:
76 |             #             best_mse = mse
77 |             #             winner = {'kp': kp, 'kd': kd, 'd': d, 'g': g, 'final_val': final_pos}
78 |
79 | # Telemetría final con el ganador para retornar datos de gráfica
80 | if winner:
81 |     model.actuator_gear[act_id, 0] = winner['g']
82 |     model.actuator_gainprm[act_id, 0] = winner['kp']
83 |     model.actuator_biasprm[act_id, 1] = -winner['kp']
84 |     model.actuator_biasprm[act_id, 2] = -winner['kd']
85 |     model.dof_damping[dof_id] = winner['d']
86 |     TARGET_SWITCH_TIME = 1.5
87 |     mujoco.mj_resetData(model, data)
88 |     t_log, p_log, f_log = [], [], []
89 |     finger_logs = {jname: [] for jname in finger_ids}
90 |     target_original = target_dict[act_name]
91 |     while data.time < sim_time:
92 |         data.ctrl[act_id] = target_original # if data.time < TARGET_SWITCH_TIME else 0.0
93 |         mujoco.mj_step(model, data)
94 |         t_log.append(data.time)
95 |         p_log.append(data.qpos[qpos_adr])
96 |         f_log.append(data.actuator_force[act_id])
97 |         for jname, qadr in finger_ids.items():
98 |             finger_logs[jname].append(data.qpos[qadr])
99 |
100 |     print(f"[SUCCESS] {act_name} optimizado.", flush=True)
101 |     target_log = [target_original for t in t_log] # if t < TARGET_SWITCH_TIME else 0.0
102 |     return {
103 |         'act_name': act_name, 'winner': winner, 'mse': best_mse,
104 |         't': t_log, 'p': p_log, 'f': f_log, 'target': target_log,
105 |         'finger_logs': finger_logs,
106 |         'finger_ranges': {
107 |             jname: (model.jnt_range[mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, jname), 0],
108 |                     model.jnt_range[mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, jname), 1])
109 |             for jname in finger_ids
110 |         }
111 |     }
112 | else:
113 |     print(f"[FAILED] {act_name} sin solución.", flush=True)
114 |     return {'act_name': act_name, 'winner': None}
115 |
116 | if __name__ == "__main__":
117 |     actuators_to_test = ["act_LeftShoulderArm", "act_Neck", "act_HipBody", "#act_BaseHip"
118 |                          "act_HeadBase", "act_HeadRot", "act_RightShoulderArm",
119 |                          "act_LeftForearm", "act_RightForearm", "act_LeftWrist",
120 |                          "act_RightWrist",
121 |                          "act_LeftLever_Slider", "act_RightLever_Slider"]
122 |
123 |     # Espacio de búsqueda
124 |     kp_values = np.arange(2000, 3500, 250)
125 |     kd_values = np.arange(0, 60, 20)
126 |     damping_values = np.arange(0, 60, 10)
127 |     gears = [1, 5]
128 |
129 |     target_val = {
130 |         # "act_BaseHip": 0.12}
131 |         "act_HipBody": 0.40, "act_Neck": 0.50,
132 |         "act_HeadBase": 0.50, "act_HeadRot": 0.50, "act_LeftShoulderArm": 0.07,
133 |         "act_RightShoulderArm": 0.20, "act_LeftForearm": 1.0, "act_RightForearm": 0.5,
134 |         "act_LeftWrist": 1.0, "act_RightWrist": 1.0,
135 |         "act_LeftLever_Slider": 0.012, "act_RightLever_Slider": 0.012,
136 |     }
137 |
138 |
139 |
140 |     sim_time = 3.2
141 |     error_threshold = 0.03
142 |
143 |     N_WORKERS = max(1, mp.cpu_count() - 1)
144 |     start_time = time.time()
145 |     print(start_time)
146 |     worker_func = partial(
147 |         search_one_actuator,
148 |         kp_values=kp_values, kd_values=kd_values, damping_values=damping_values,
149 |         gears=gears, target_dict=target_val, sim_time=sim_time, error_threshold=error_threshold
150 |     )
151 |
152 |     with mp.Pool(N_WORKERS) as pool:
153 |         results = pool.map(worker_func, actuators_to_test)
154 |
155 |     # Filtrar solo los exitosos para graficar
156 |     valid_results = [r for r in results if r['winner'] is not None]
157 |     valid_results.sort(key=lambda x: x['mse'])
158 |     end = (time.time() - start_time)
159 |

```

```

160 |     if valid_results:
161 |         display_count = len(valid_results)
162 |         cols = 2
163 |         rows = int(np.ceil(display_count / cols))
164 |         fig, axs = plt.subplots(rows, cols, figsize=(15, rows * 4))
165 |         axs = axs.flatten() if display_count > 1 else [axs]
166 |
167 |         for i, res in enumerate(valid_results):
168 |             has_fingers = bool(res.get('finger_logs'))
169 |             ax = axs[i]
170 |             w = res['winner']
171 |             ax.plot(res['t'], [res['target']] * len(res['t']), 'r--', label='Target')
172 |             ax.plot(res['t'], res['p'], 'b-', label='Posición')
173 |             if has_fingers:
174 |                 ax.set_title(f"{res['act_name']} - Gear:{w['g']} | KP:{w['kp']} | KD:{w['kd']} | D:{w['d']}",
fontsize=9)
175 |             else:
176 |                 ax.set_title(f"{res['act_name']} - Gear:{w['g']} | KP:{w['kp']} | KD:{w['kd']} | D:{w['d']}",
fontsize=9)
177 |
178 |             if has_fingers:
179 |                 colors = ['darkorange', 'purple']
180 |                 styles = ['-', '--']
181 |                 for (jname, flog), col, sty in zip(res['finger_logs'].items(), colors, styles):
182 |                     lo, hi = res['finger_ranges'][jname]
183 |                     ax.plot(res['t'], flog, color=col, ls=sty, lw=1.2,
label=f"{jname.split('_')[0]+jname.split('_')[1][-3:]} (rad)")
184 |                     # línea de límite físico del finger
185 |                     limit_val = hi if flog[-1] > 0 else lo
186 |                     ax.axhline(limit_val, color=col, ls=':', lw=0.8, alpha=0.6, label=f"límite
{jname.split('_')[0]}[-1]+jname.split('_')[1][-3:]}: {limit_val:.3f}")
187 |                     # Eje secundario para fuerza
188 |                     ax2 = ax.twinx()
189 |                     ax2.plot(res['t'], res['f'], 'g-', alpha=0.3, label='Fuerza')
190 |
191 |                     ax.set_title(f"{res['act_name']} - Gear:{w['g']} | KP:{w['kp']} | KD:{w['kd']} | D:{w['d']}")
192 |                     ax.grid(True, alpha=0.3)
193 |
194 |         plt.tight_layout()
195 |         plt.show()
196 |
197 |         # Tabla Resumen
198 |         summary = []
199 |         for r in valid_results:
200 |             summary.append({'Actuador': r['act_name'], **r['winner'], 'MSE': r['mse']})
201 |             print("\n" + "="*60)
202 |             print(pd.DataFrame(summary).to_string(index=False))
203 |             print("="*60)
204 |         else:
205 |             print("No se encontraron configuraciones válidas.")
206 |         print(f'Total Time: {end}')

```